



KE&M II: Ontology

Hutchatai Chanlekha

Department of Computer Engineering, Kasetsart University





OWL ontology: Elements

- OWL: Web Ontology Language
- Most of the elements of an OWL ontology concern:
 - Classes
 - Properties (or Relations)
 - Instances of classes (i.e. individual)
 - Relationships between these instances.
- Distinction between a *class* and an *individual* in OWL

Class

- A name and collection of properties that describe a set of individuals

Individual

- The members of those classes
- Correspond to actual entities that can be grouped into these classes

OWL ontology: Class VS Individual

- In building ontologies, class and individual distinction is frequently blurred:
- Class vs. Individual (Instance)***
 - In certain contexts, something that is obviously a class can itself be considered an instance of something else.



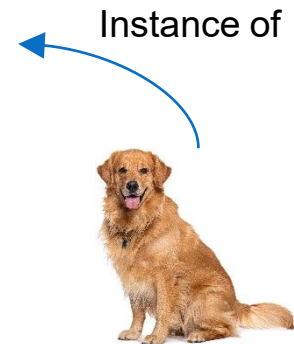
Height	21.5–24 inches
Weight	55–75 pounds
Lifespan	10–12 years
Breed group	Sporting
Shed level	High
Temperament	Friendly
Energy level	Active
Common health concerns	Hot spots, atopic dermatitis, hypothyroidism, ear infections, hip dysplasia

as an instance if the most specific thing your ontology must be able to answer is a dog breed.



Height	21.5–24 inches
Weight	55–75 pounds
Lifespan	10–12 years
Breed group	Sporting
Shed level	High
Temperament	Friendly
Energy level	Active
Common health concerns	Hot spots, atopic dermatitis, hypothyroidism, ear infections, hip dysplasia

As a class if the most specific thing that your ontology must be able to answer is an individual dog.





OWL ontology: Property

- Property
 - *Properties* let us assert general facts about the concepts/classes and specific facts about individuals
 - Two types of property
 - *data properties*: classes to datatypes (or individuals to datatypes)
 - *object properties*: relate classes to classes (or individuals to individuals)

```
<owl:Class rdf:ID="Wine" />  
<owl:Class rdf:ID="Grape" />  
<owl:Class rdf:ID="VintageYear" />
```

```
<owl:ObjectProperty rdf:ID="MadeFrom">  
  <rdfs:domain rdf:resource="#Wine" />  
  <rdfs:range rdf:resource="#Grape" />  
</owl:ObjectProperty>
```

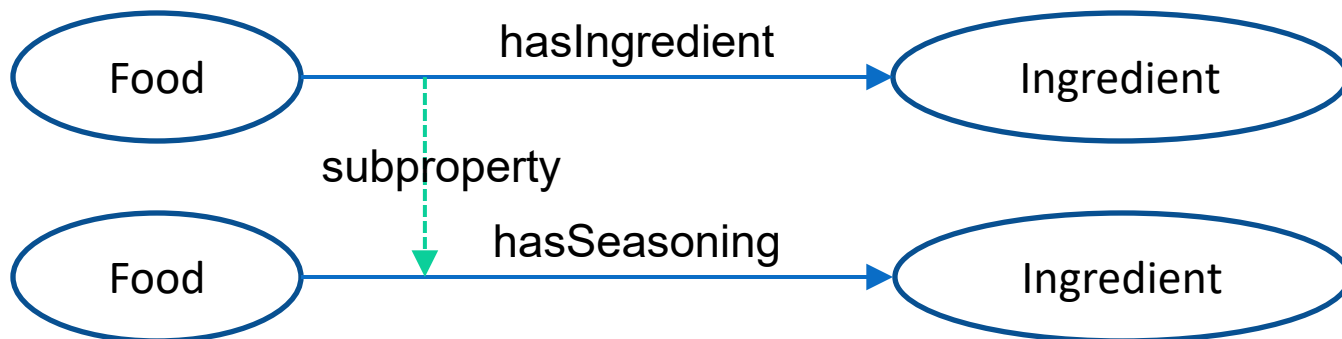
```
<owl:DatatypeProperty rdf:ID="yearValue">  
  <rdfs:domain rdf:resource="#VintageYear" />  
  <rdfs:range rdf:resource="&xsd; positiveInteger"/>  
</owl:DatatypeProperty>
```

OWL ontology: Property hierarchy [1]

- Properties, like classes, can be arranged in a hierarchy.

```
<owl:ObjectProperty rdf:ID="hasIngredient">  
  <rdfs:domain rdf:resource="#Food" />  
  <rdfs:range rdf:resource="#Ingredient" />  
</owl:ObjectProperty>
```

```
<owl:ObjectProperty rdf:ID="hasSeasoning">  
  <rdfs:subPropertyOf rdf:resource="#hasIngredient" />  
</owl:ObjectProperty>
```



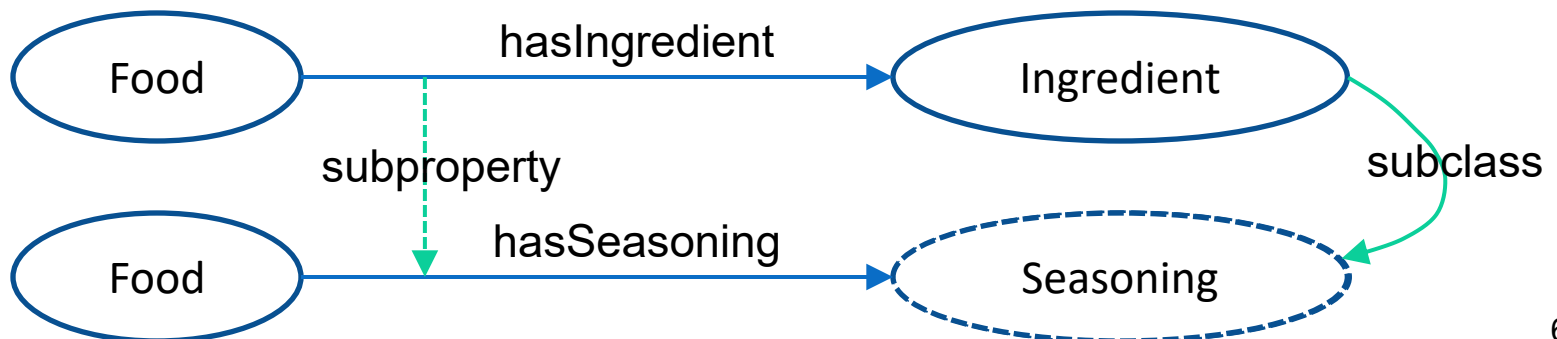


OWL ontology: Property hierarchy [2]

- Properties, like classes, can be arranged in a hierarchy.

```
<owl:ObjectProperty rdf:ID="hasIngredient">  
  <rdfs:domain rdf:resource="#Food" />  
  <rdfs:range rdf:resource="#Ingredient" />  
</owl:ObjectProperty>
```

```
<owl:ObjectProperty rdf:ID="hasSeasoning">  
  <rdfs:subPropertyOf rdf:resource="#hasIngredient" />  
  <rdfs:range rdf:resource="#Seasoning" />  
</owl:ObjectProperty>
```



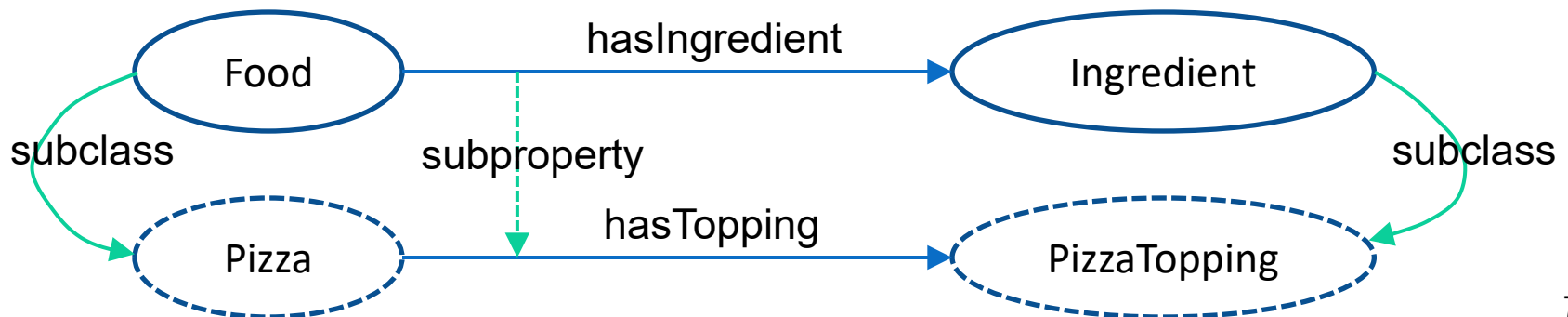


OWL ontology: Property hierarchy [3]

- Properties, like classes, can be arranged in a hierarchy.

```
<owl:ObjectProperty rdf:ID="hasIngredient">  
  <rdfs:domain rdf:resource="#Food" />  
  <rdfs:range rdf:resource="#Ingredient" />  
</owl:ObjectProperty>
```

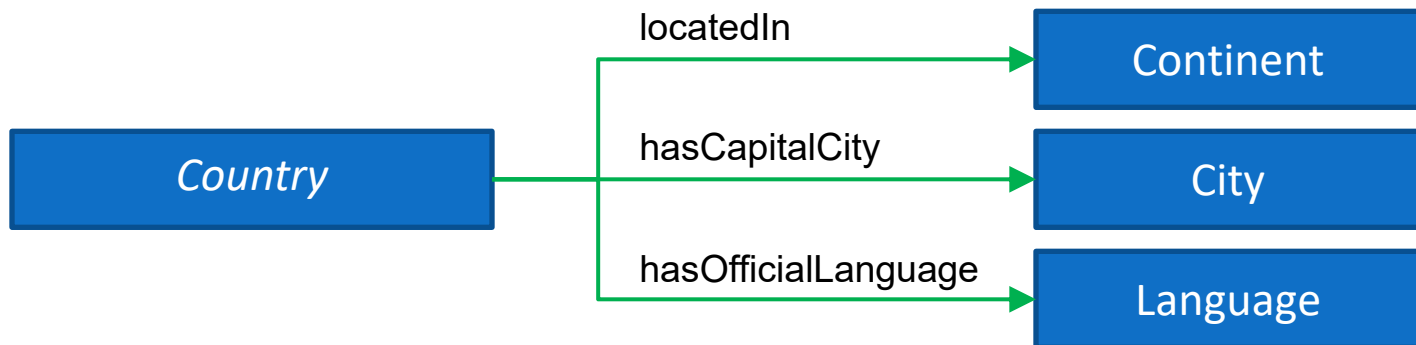
```
<owl:ObjectProperty rdf:ID="hasTopping">  
  <rdfs:subPropertyOf rdf:resource="#hasIngredient" />  
  <rdfs:domain rdf:resource="#Pizza" />  
  <rdfs:range rdf:resource="#PizzaTopping" />  
</owl:ObjectProperty>
```





OWL ontology: Property of individual

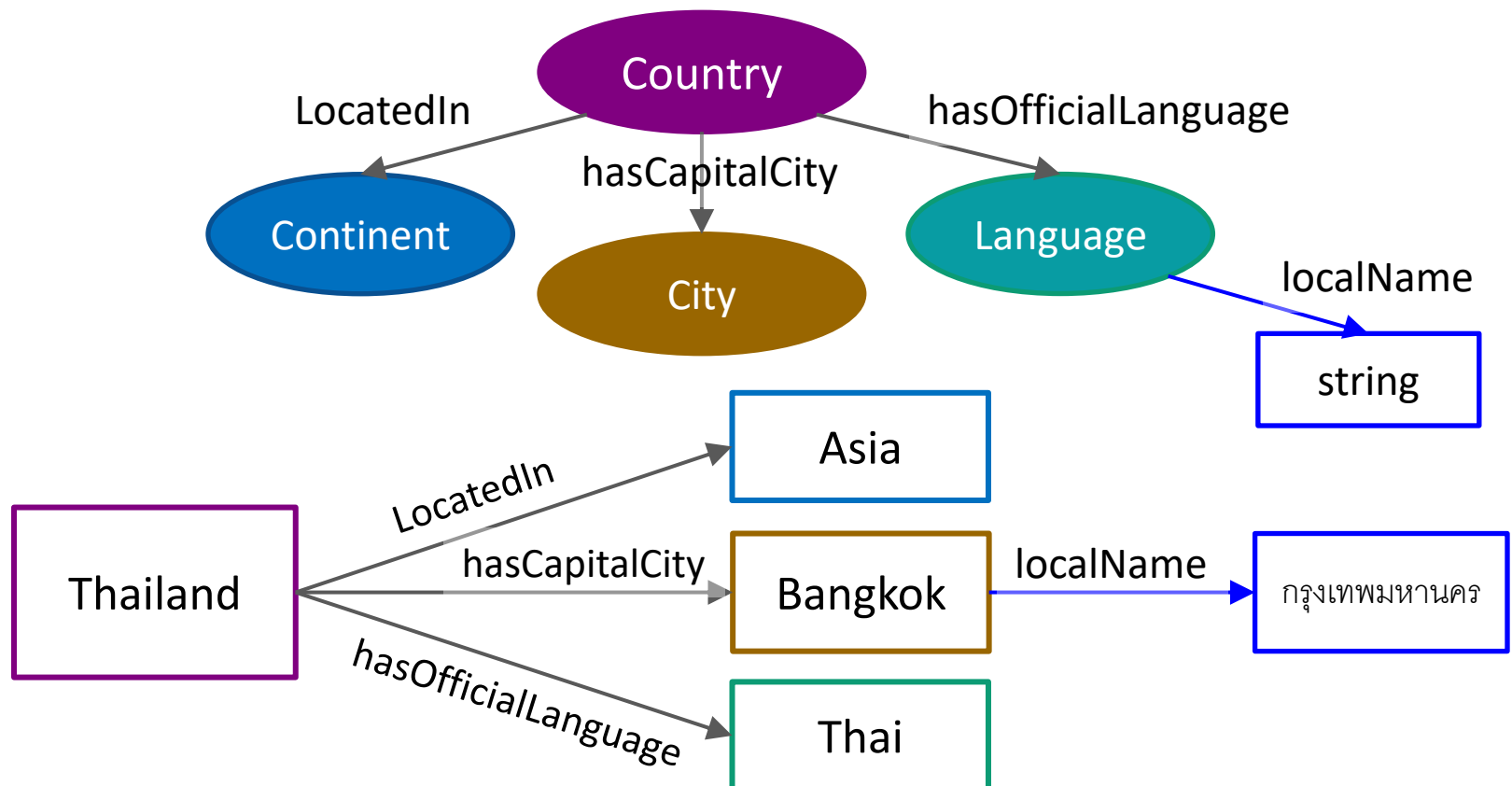
- Suppose *Country* class can have properties:
 - locatedIn where the value is the class *Continent*
 - hasCapitalCity where the value is the class *City*
 - hasOfficialLanguage where the value is the class *Language*



- *Thailand* is an instance of the class *Country*
- To add these three property values for *Thailand*, you have to create instances for the class *Continent*, *City*, and *Language* first.

OWL ontology: Property of individual

- Property of individual
 - Suppose we know that Thailand is located in Asia. Its capital city is Bangkok, and its official language is Thai.





OWL ontology: Property of individual

- Property of individual:
 - Suppose we know that Thailand is located in Asia. Its capital city is Bangkok, and its official language is Thai.

```
<City rdf:ID="Bangkok" />  
<localName rdf:datatype="&xsd:string">กรุงเทพมหานคร</localName>  
</City>
```

```
<Continent rdf:ID="Asia" />  
<Language rdf:ID="Thai">
```

Define Continent,
City, and Language
individuals to be used
as property values

```
<Country rdf:ID="Thailand" >  
  <locatedIn rdf:resource="#Asia"/>  
  <hasCapitalCity rdf:resource="#Bangkok" />  
  <hasOfficialLanguage rdf:resource="#Thai">  
</Country>
```

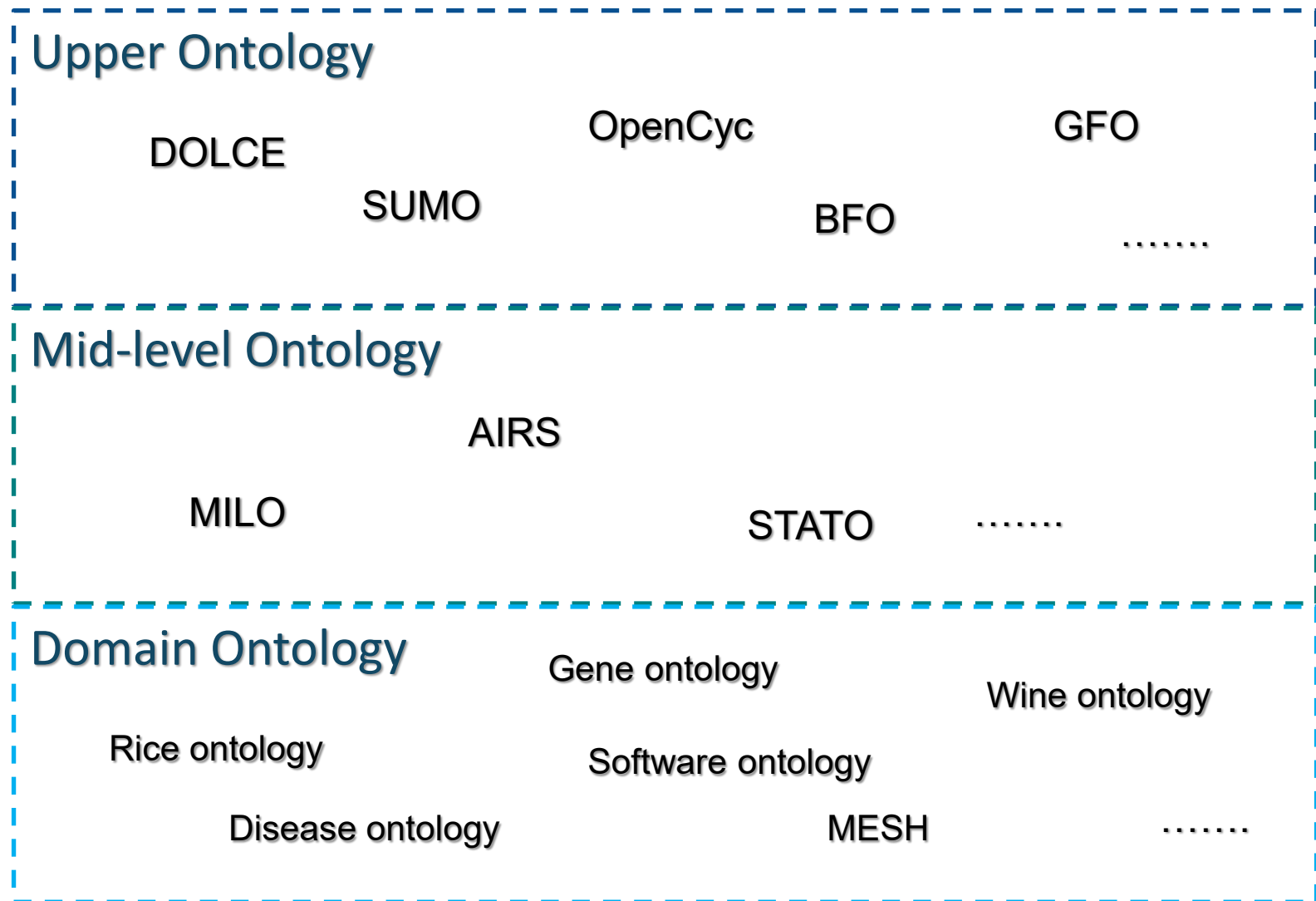
Country
individual



BROAD CLASSES OF ONTOLOGIES



Broad classes of Ontologies





Upper Ontology

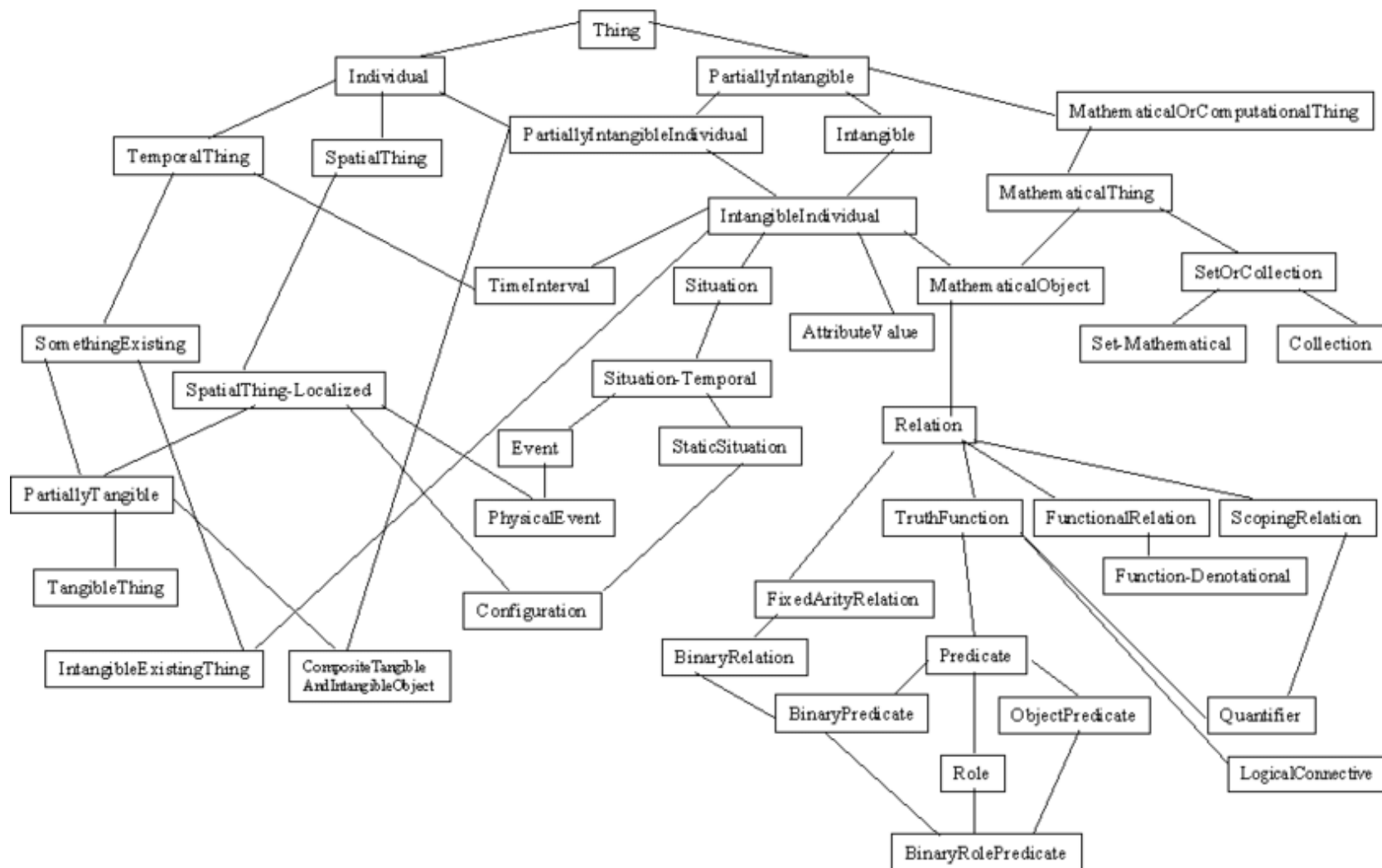
- Upper ontology or Foundational ontology or Universal ontology
 - Generic ontologies applicable to various domains
 - Define basic notions such as objects, relations, events, process, etc.
- An upper ontology is a high-level, domain-independent ontology
 - Provide a domain independent conceptual model that aims to be highly re-usable across specific domain applications
 - Often characterized as representing common sense concepts, i.e. those that are basic for human understanding of the world
- The concepts expressed in upper ontologies are intended to be basic and universal concepts
 - Usually describe very general level or abstract concepts
 - Ensure generality and expressivity for a wide area of domains.



Upper Ontology (cont.)

- Provide a framework by which disparate systems may utilize
 - One of the primary purposes of upper ontologies is to aid semantic integration across ontologies and to encourage a set of design principles within those ontologies that use them
- Can be used for deriving more domain-specific ontologies
- Domain or core reference ontologies based on the same foundational ontology can be more easily integrated
- More information on upper ontologies
 - A Comparison of Upper Ontologies
 - <http://www.disi.unige.it/person/MascardiV/Download/DISI-TR-06-21.pdf>

Example of OpenCyc





Mid-Level Ontology

- Serves as a bridge between abstract concepts defined in the upper ontology and low-level domain specific concepts specified in a domain ontology.
 - May provide more concrete representations of abstract concepts found in the upper ontology.
- Mid-level (as well as upper) ontologies are intended to provide a mechanism to make this mapping of concepts across domains easier.
- Represent knowledge at an intermediate level of detail independently of a specific task
- This ontology category also includes the set of ontologies that represent commonly used concepts, such as Time and Location.



Domain Ontology

- Specifies concepts particular to a domain of interest and represents those concepts and their relationships from a domain specific perspective.
- Domain ontology is only applicable to a domain with a specific view point.
 - View point defines how a group of users conceptualize and visualize some specific phenomenon
- Could be linked to a specific application



Domain Ontology

- Domain ontologies may be composed by importing mid-level ontologies. They may also extend concepts defined in mid-level or upper ontologies.
 - Reusing well-established ontologies in a domain ontology development allows one to take advantage of the semantic richness of the relevant concepts and logic already built into the reused ontology
 - Using common mid-level and upper ontologies can ease the process of integrating or mapping domain ontologies.
- Ontologies designed for specific tasks are called “*application ontologies*”



Local Ontologies/Application Ontologies

- Specialization of domain ontologies
 - There could be NO consensus or knowledge sharing
- Represents the particular model of a domain according to a single viewpoint of a user or a developer.



Reference

- Slides based on
 - **An Introduction to Ontologies and Ontology Engineering**
 - Catherine Roussey, Francois Pinet, Myoung Ah Kang, Oscar Corcho
 - http://oa.upm.es/10381/1/An_Introduction.pdf
 - **W3C OWL guide**
 - <http://www.w3.org/TR/owl-guide/>
 - <http://www.slideshare.net/marinasantini1/09-semantic-webontologies>